HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

TRƯỜNG ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

LÝ THUYẾT MẬT MÃ

TS. TẠ KIM HUỆ
TRƯỜNG ĐIỆN – ĐIỆN TỬ

ONE LOVE. ONE FUTURE.

CHƯƠNG 2: MẬT MÃ KHOÁ ĐỐI XỨNG

- Mật mã học hiện đại và các khái niệm
- Mật mã dòng
- Mật mã khối

Things to remember

- Mật mã học là:
  - A tremendous tool : công cụ mạnh
  - The basis for many security mechanisms: cội nguồn của rất nhiều kỹ thuật bảo mật khác
- Mật mã học không:
  - The solution to all security problems: giải quyết được tất cả các vấn đề bảo mật
  - Reliable unless implemented and used properly: đáng tin cậy trừ khi sử dụng đúng cách và thực thi đúng cách
- Tự sáng tạo ra hệ mật mã của riêng bạn
  - many many examples of broken ad-hoc design
- Encryption algorithm is publicly known -> Thuật toán mật mã luôn công khai -> Không sử dụng các loại mã độc quyền (Never use a proprietary cipher)

Mật mã học ứng dụng ở khắp mọi nơi

Bảo mật truyền thông (Secure communication):

- web traffic: HTTPS
- wireless traffic: 802.11i WPA2 (and WEP), GSM, Bluetooth

Mã hóa file (Encrypting files on disk): EFS, TrueCrypt

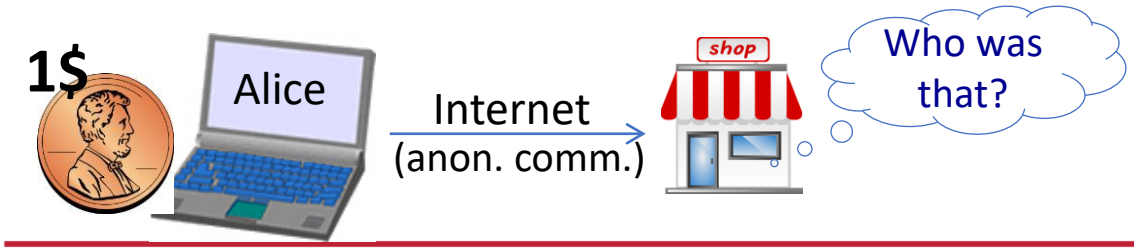
Bảo vệ nội dung (Content protection) (e.g. DVD, Blu-ray): CSS, AACS

Xác thực người dùng (User authentication)

... and much much more

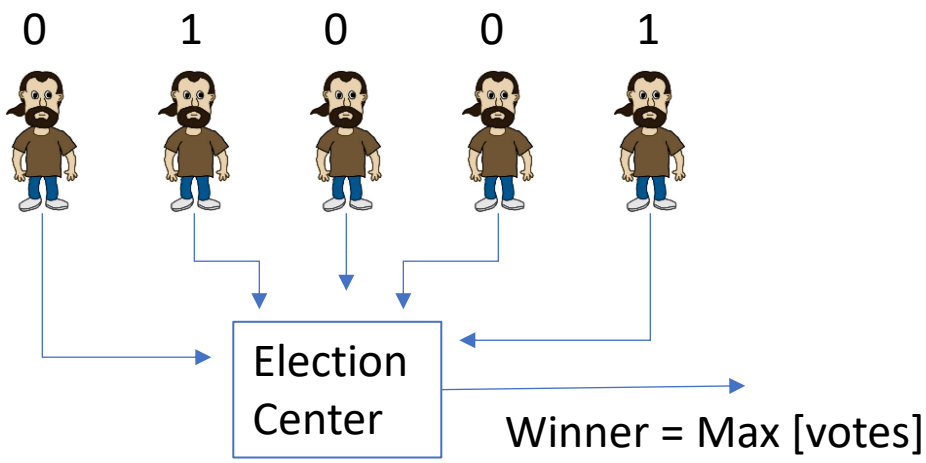
Mật mã học ứng dụng ở khắp mọi nơi

- Chữ ký số (Digital signatures)
- Giao tiếp ẩn danh (Anonymous communication)
- Tiền số ẩn danh (Anonymous **digital** cash)
  - Can I spend a “digital coin” without anyone knowing who I am?
  - How to prevent double spending?



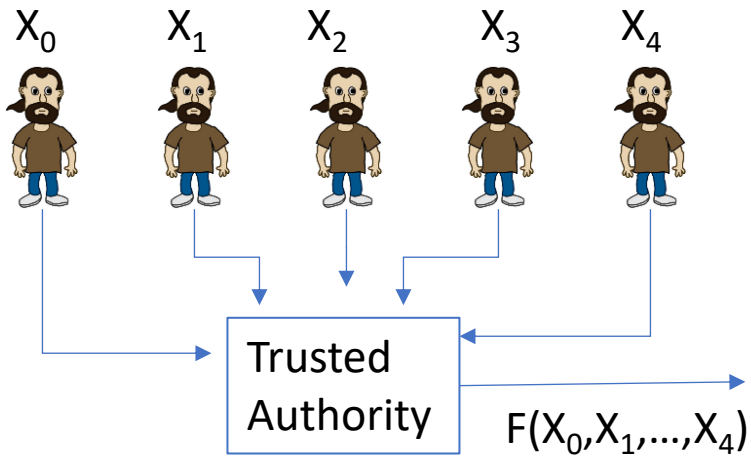
Đảm bảo tính chính xác trong các giao thức

- Bầu cử (Elections)
- Đấu giá riêng tư (Private auctions)
- Auction winner = [the highest bidder, pays the 2nd highest bit]



Đảm bảo tính chính xác trong các giao thức

- Goal: compute $f(x_0, x_1, x_2, x_3, x_4)$
- "Thm:" anything that can done with trusted auth. can also be done without
- Tính toán bảo mật nhiều bên (Secure multi-party computation)



Mật mã học hiện đại – Các khái niệm

- Một hệ thống mật mã là một bộ năm

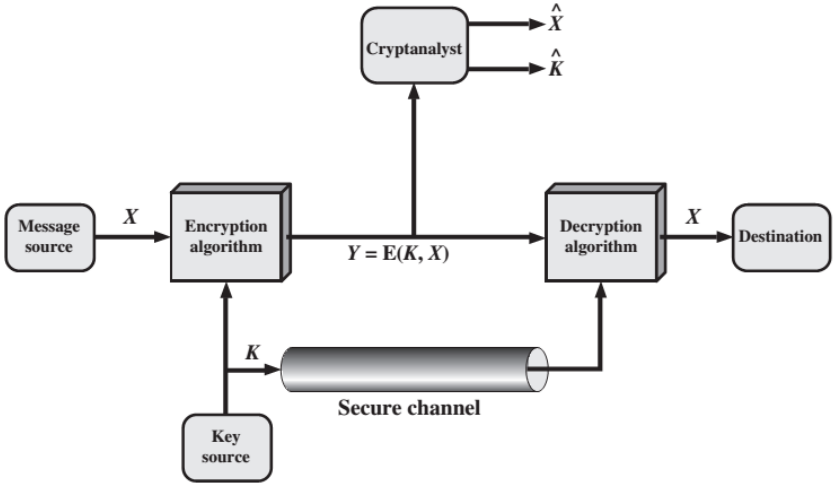
$$\boldsymbol{S} = (\boldsymbol{P}, \boldsymbol{C}, \boldsymbol{K}, \boldsymbol{E}, \boldsymbol{D})$$

Thỏa mãn các điều kiện sau đây:

- Tập nguồn P (plaintext) là tập hữu hạn tất cả các bản tin nguồn cần mã hóa có thể có.
- C (ciphertext) là một tập hữu hạn các ký tự bản mã
- K (key) là tập hữu hạn các khóa có thể được sử dụng
- E (encryption) là một ánh xạ từ (K,P) vào C, được gọi là phép lập mật mã
- D (decryption)là một ánh xạ từ (K,C) vào P , được gọi là phép giải mã.
- *Chú ý: các ánh xạ này là ánh xạ 1-1*

Mật mã học hiện đại – Các khái niệm

- Với mỗi khóa $\boldsymbol{k} \in \boldsymbol{K}$, tồn tại luật mật mã $\boldsymbol{e_k} \in \boldsymbol{E}$ và luật giải mật mã $\boldsymbol{d_k} \in \boldsymbol{D}$ tương ứng.
- Luật mật mã $\boldsymbol{e_k}$: $\boldsymbol{P} \rightarrow \boldsymbol{C}$ và luật giải mật mã $\boldsymbol{d_k}$: $\boldsymbol{C} \rightarrow \boldsymbol{P}$ là hai ánh xạ thỏa mãn: $\boldsymbol{d_k}\big(\boldsymbol{e_k}(\boldsymbol{x})\big) = \boldsymbol{x}, \forall \boldsymbol{x} \in \boldsymbol{P}$



Mật mã khối | Mật mã dòng | Hàm băm – chữ ký số - xác thực

Khóa đối xứng – Khóa bất đối xứng

Mã sử dụng khóa – không sử dụng khóa

Mật mã khoá đối xứng

<u>Def:</u> Một hệ mã mật được định nghĩa bởi: $(\mathcal{K} \times \mathcal{M} \rightarrow \mathcal{C})$

$$E: \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{C} \text{ và } D: \mathcal{K} \times \mathcal{C} \rightarrow \mathcal{M}$$

$$\forall\, m \in \mathcal{M}, k \in \mathcal{K}: D\big(k, E(k, m)\big) = m$$

- ***E*** sẽ thường là ngẫu nhiên.
- ***D*** sẽ luôn xác định.

Thế nào là mã mật bảo mật?

- Khả năng tấn công (Attacker's abilities): Chỉ tấn công bản mã (**CT only attack)** (for now)
- Yêu cầu bảo mật khả thi (Possible security requirements):
- Tấn công 1: Kẻ tấn không thể khôi phục khóa mật
- Tấn công 2: Kẻ tấn công không thể khôi phục toàn bộ bản rõ

$E(k, m) = m$ -> sẽ an toàn ?

$E(k, m_0||m_1) = m_0||k \oplus m_1$ -> sẽ an toàn ?

Lý thuyết bảo mật thông tin (Shannon 1949)

Def: Hệ mật mã (***E***, ***D***) trong tập $(\mathcal{K}, \mathcal{M}, \mathcal{C})$ là bí mật hoàn hảo (**perfect secrecy) nếu**

$\forall\, m_0, m_1 \in \mathcal{M}, [len|m_0| = len|m_1|]$ và $\forall c \in \mathcal{C}$ thì

$\Pr[E(k, m_0) = c] = \Pr[E(k, m_1) = c]$

Khi đó k là đồng nhất trong tập $\mathcal{K}$ ($k \xleftarrow{R} \mathcal{K}$)

- Bản mã sẽ không thể hé lộ thông tin về *m* nếu *m* là m_0 hoặc m_1 , tất cả các lợi thế tấn công mạnh nhất đều không thể có được PT từ CT, do đó tấn công từ CT sẽ không khả thi (tuy nhiên các kiểu tấn công khác thì có thể)

Mật mã dòng

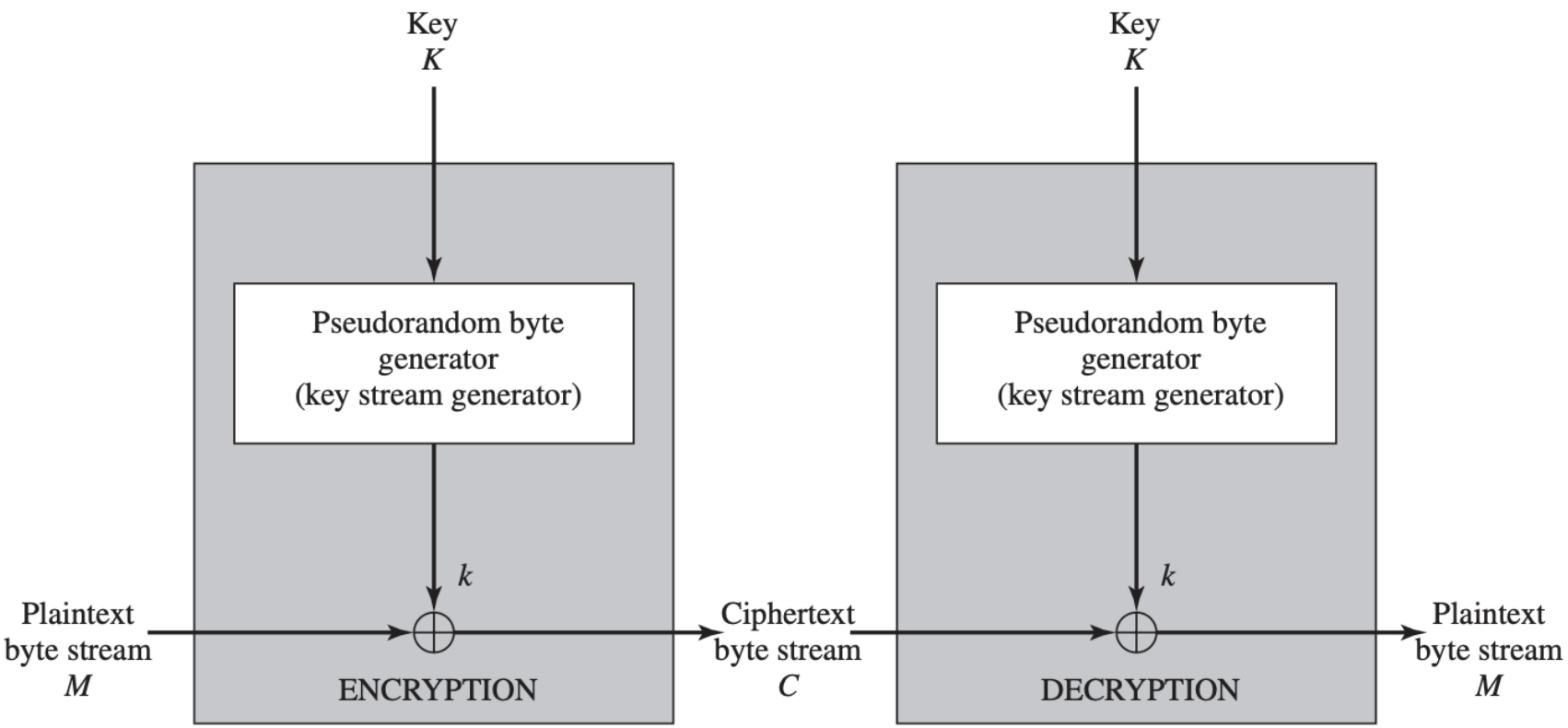


Figure 7.5 Stream Cipher Diagram

The One Time Pad (OTP) (Vernam 1917)

- Bảng mã sử dụng một lần
- Ví dụ đầu tiên của một mã mật bảo mật (“secure” cipher), key = một chuỗi bit ngẫu nhiên bằng với chiều dài bản tin (random bit string as long the message), CT = cipher text
- $\mathcal{M} = \mathcal{C} = [0,1]^n, \mathcal{K} = [0,1]^n$
- $\mathcal{C} = E(\mathcal{K}, \mathcal{M}) = \mathcal{K} \oplus \mathcal{M}$
- $D(\mathcal{K}, C) = \mathcal{K} \oplus \mathcal{C}$

Biến đổi

$D(\mathcal{K}, C) = D(\mathcal{K}, E(\mathcal{K}, \mathcal{M}))$

$= \mathcal{K} \oplus (\mathcal{K} \oplus \mathcal{M})$

$= \mathcal{K} \oplus \mathcal{K} \oplus \mathcal{M} = \mathcal{M}$

msg: 0 1 1 0 1 1 1

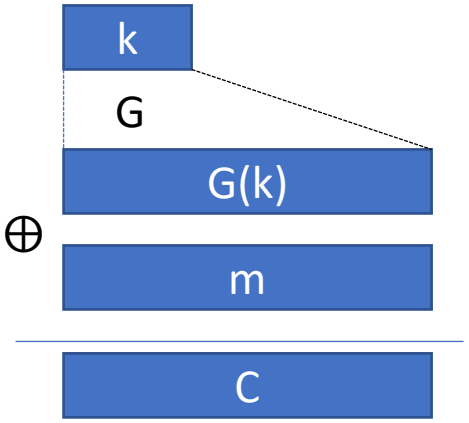
key: 1 0 1 1 0 1 0 $\oplus$

CT:

Bộ phát số ngẫu nhiên - Pseudorandom Generators

- Ý tưởng: thay thế khoá ngẫu nhiên bằng ("random" key) bằng khoá giả ngẫu nhiên
- PRG là một ánh xạ G: $\underbrace{[0,1]^s}_{seed\ space} \longrightarrow [0,1]^n, n \gg s$
- Các phép toán thông qua thuật toán xác định (deterministic algorithm)

$$C = E(k,m) = m \oplus G(k)$$
$$D(k,C) = C \oplus G(k)$$



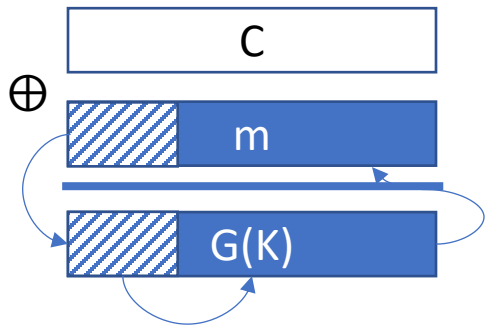
PRG không thể dự đoán (unpredictable)

- Giả sử PRG là dự đoán được $\exists i: G(k)|_{1,2,\ldots,i} \rightarrow G(k)|_{i+1,i+2,\ldots,n}$ do đó
- Mặc dù $G(k)|_{1,2,\ldots,i} \rightarrow G(k)|_{i+1,i+2,\ldots,n}$ là một trở ngại (problem)!
- Có thể khẳng định G: K ⟶ $\{0,1\}^n$ là dự đoán được, nếu:
- ∃ "eFF" $alg. A\ and\ \exists\ 0\ \leq\ i\ \leq\ n-1$
- Khi có $Pr_{k \underset{R}{\leftarrow} \mathcal{K}}[A(G(k))|_{1,2,\ldots,i} = G(k)|_{i+1}\ ]\ > 1/2 + \mathcal{E}$
- Cho một đáng kể (non-negligible) $\mathcal{E}$ (e.g., $\mathcal{E}$= $1/2^{30}$)

PRG không thể dự đoán (unpredictable)

<u>Def</u>: PRG là không thể dự doán nếu nó không có khả năng dự đoán

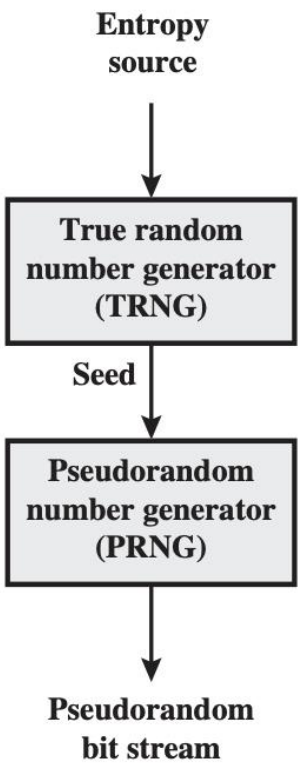
$\Rightarrow$ $\forall i$: không có một lợi thế đáng kể nào ("eff" adv.) có thể dự đoán (i+1) tương ứng với "non-neg" ε



- Giả sử G: K $\longrightarrow$ $\{0,1\}^n$ do đó tất cả k: $\boldsymbol{XOR(G(k)) = 1}$
- Thì G có thể đoán trước được không ?? -> Có

Chuỗi PRG yếu

- Bộ tạo đồng dư tuyến tính (Linear Congruential Generators)
- m giá trị modulus (modulus) $m > 0$
- a số nhân (multiplier) $0 < a < m$
- c số hạng (increment) $0 \leq c < m$
- X_0 giá trị khởi tạo (starting value), seed (hạt mầm)
- Chuỗi số giả ngẫu nhiên được tạo như sau
  $X_{n+1} = (aX_n + c) mod\ m$



Generation of Seed Input to PRNG

Ví dụ

- Giá trị $a = c = 1$
- Giá trị $a = 7, c = 0, m = 32$ và $X_0 = 1$
- Tìm chuỗi được tạo

  - Đáp án:
  - Chuỗi {7,17,23,1,7,etc. } -> chỉ 4 giá trị được sử dụng chu kỳ là 4
  - Thay đổi a = 5 , chu kỳ tăng lên 8

- Điều kiện m lớn thỏa mãn 3 tiêu chuẩn sau
  - Chuỗi được tạo ra phải chứa tất cả các số từ 1-m trước khi lặp lại (full-period)
  - Chuỗi phải ngẫu nhiên
  - Thực thi hiệu quả với số 32 bit

Chuỗi PRG yếu

- Thuật toán tạo chuỗi số giả ngẫu nhiên thông qua bộ tạo đồng dư tuyến tính (Linear congruential generator) với các tham số a, b, p :
- $r[i] \leftarrow a.r[i-1] + b \mod p$
- Output bit of $r[i]$
- $i++$
- $Seed \equiv r(0)$
- glibc $random()$:
- $r[i] \leftarrow (r[i-3] + r[i-31])\ \% = 2^{32}$
- output $r[i] >> 1$
- Never use $random(\quad)$ for crypto!! (e.g. Kerberos V4)

Chuỗi PRG yếu

- Bộ tạo Blum blum blum
- Chọn 2 số nguyên tố lớn p, q đồng dư 3 trong phép chia cho 4
- $p \equiv q \equiv 3 \ (mod\ 4)$ có nghĩa là
- $p\ mod\ 4 = q\ mod\ 4 = 3$
- Ví dụ $p = 7, q = 11$. Tính $n = p \times q$
- Chọn một số ngẫu nhiên s, thực tế là s nguyên tố cùng nhau với n , điều này có nghĩa là p, q không phải là nhân tử của s
- Giải thuật như sau: $X_0 = s^2\ mod\ n$

For $i = 1$ to ∞ do $X_i = {X_{i-1}}^2\ mod\ n\ ; B_i = X_i\ mod\ 2$

Bài tập

- Cho n = 192649, tìm p , q , s và chuỗi số được tạo ra sau 20 vòng lặp, the seed s = 101355
-

i	X_i	B_i
0	20749	
1	143135	1
2	177671	1
3	97048	0
4	89992	0
5	174051	1
6	80649	1
7	45663	1
8	69442	0
9	186894	0
10	177046	0

i	X_i	B_i
11	137922	0
12	123175	1
13	8630	0
14	114386	0
15	14863	1
16	133015	1
17	106065	1
18	45870	0
19	137171	1
20	48060	0

Mật mã dòng

- RC4
- CSS
- eStream family Salsa20, Sosemanuk, Gain 128 …
- https://en.m.wikipedia.org/wiki/ESTREAM

RC4

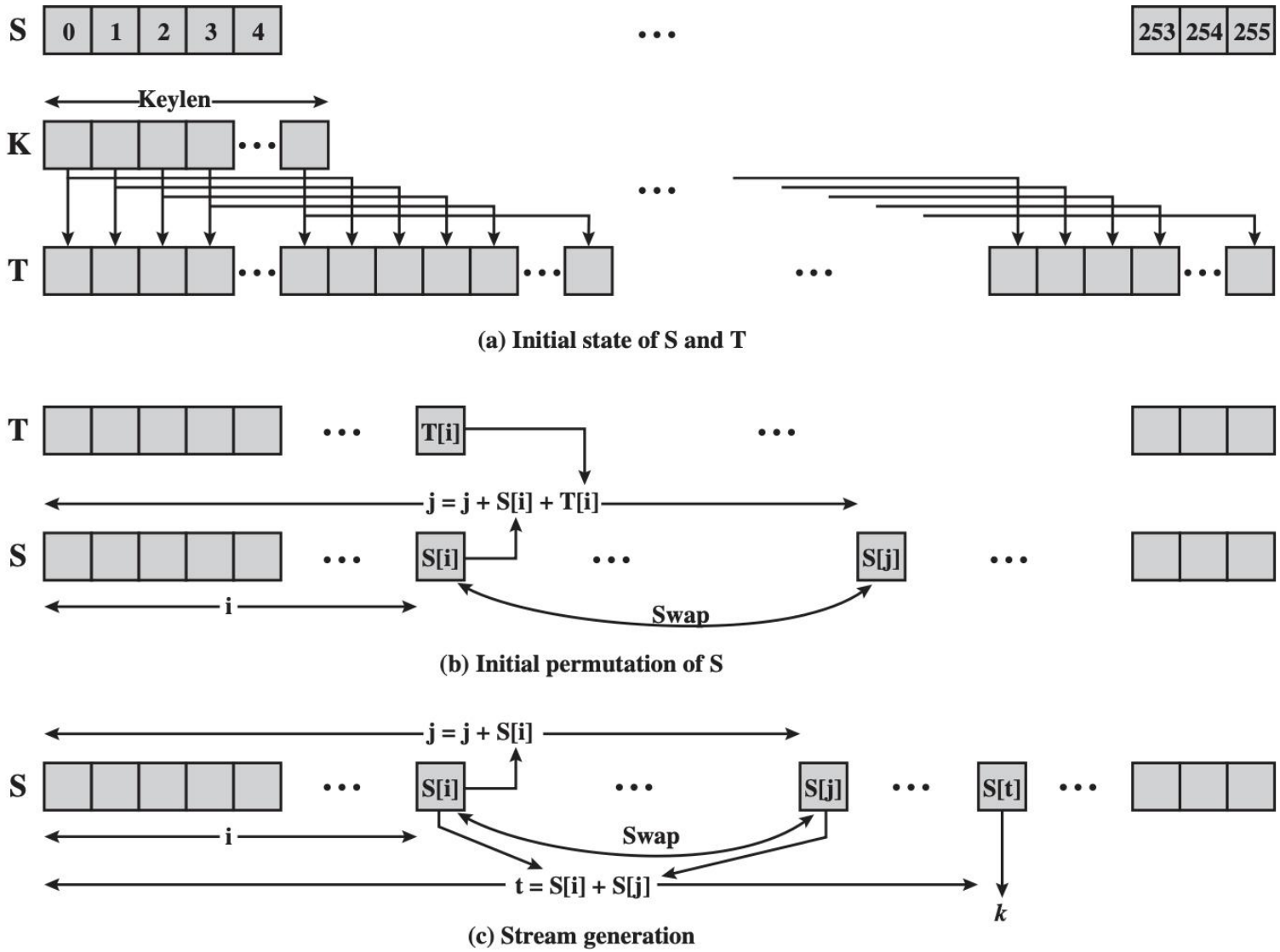
- RC4 là mật mã dòng phát triển từ năm 1987 bởi Ron Rivest (RSA Security).
- RC4 được sử dụng trong chuẩn Secure Sockets Layer/Transport Layer Security (SSL/TLS) và được tùy biến cho các dịch vụ Web browsers và servers.
- Được sử dụng trong giao thức Wired Equivalent Privacy (WEP) và giao thức truyền tin không dây WiFi Protected Access (WPA) là một phần của chuẩn 802.11 wireless LAN.

Thuật toán mã dòng RC4

- Khoá có chiều dài tuỳ biến từ 1 đến 256 bytes (8 to 2048 bits) được sử dung để tạo một vector trạng thái S với các thành phần S[0], S[1], … , S[255].
- S chứa hoán vị 8 – bit từ 0 đến 255.
- Một byte k được tạo từ S bằng việc lựa chọn 1 trong 255 thực thể của S
- Mỗi khi k được tạo ra thì các thực thể của S sẽ được hoán vị
- Các bước thực hiện như sau

Initialization -> Initial Permutation of S -> Stream Generation

Thuật toán mã dòng RC4



(a) Initial state of S and T

(b) Initial permutation of S

(c) Stream generation

Thuật toán mã dòng RC4

- Pha khởi tạo, giá trị ban đầu của vector S được mặc định $S[0] = 0, S[1] = 1, \ldots, S[255] = 255$
- Vector T được tạo ra trong trường hợp khoá K có 256 byte thì T sẽ là chuyển vị của S. Trường hợp khác, khoá K có chiều dài là $keylen$ byte, thì $keylen$ phần tử đầu tiên của T sẽ được sao chép từ K, và K sẽ được lặp lại nhiều lần cho đến khi điền đầy T.

```
/* Initialization */
for i = 0 to 255 do
S[i] = i;
T[i] = K[i mod keylen];
```

Thuật toán mã dòng RC4

- Sử dụng T để tạo ra hoán vị khởi tạo của S. Bao gồm điểm khởi tạo $S[0]$ đến $S[255]$, cho mỗi $S[i]$, đảo $S[i]$ với các byte khác của S tương ứng lược đồ đọc ra từ $T[i]$:
- Đây là phép hoán vị do đó S vẫn chứa các số từ 0 đến 255.

```
/* Initial Permutation of S */
j = 0;
for i = 0 to 255 do
j = (j + S[i] + T[i]) mod 256;
Swap (S[i], S[j]);
```

Thuật toán mã dòng RC4

- Stream Generation
- Khi vector S được khởi tạo, khoá đầu vào không đủ dài để sử dụng. Stream generation tạo ra chu kỳ mà tất cả các phần tử của S[i], từng S[i], swapping S[i] và các byte khác trong S được viết lại bởi cấu trúc S hiện tại. Sau khi đạt được S[255], quá trình xử lý được tiếp tục và bắt đầu lại từ S[0]
- Để mã hoá, toán tử XOR sẽ được dùng để tác động k và byte kế tiếp của bản rõ

```
/* Stream Generation */ i, j = 0;
while (true)
i = (i + 1) mod 256;
j = (j + S[i]) mod 256;
Swap (S[i], S[j]);
t = (S[i] + S[j]) mod 256; k = S[t];
```

Bài tập

- Cho K = [1,2,3,6] , biết vector S là một chuỗi byte được biểu diễn từ 0 đến 9
- Tìm Chuỗi khóa được tạo ra bởi thuật toán RC4, biết chiều dài plaintext là 6

Bài tập

- Mã hoá chuỗi thông tin sau sử dụng RC4

"Hanoi University of Science and Technology"

Câu hỏi về RC4

- Giá trị chuỗi khoá của RC4 như thế nà sẽ làm cho S giữ nguyên sau quá trình khởi tạo hoán vị (Initial Permutation)? Nghĩa là, sau hoán vị ban đầu của S, các phần tử của S sẽ bằng các giá trị từ 0 đến 255 theo thứ tự tăng dần.
- Cần bao nhiêu bit để lưu trữ giá trị trong ở trạng thái inital permutation ? (biến đếm i,j, và các byte trạng thái)
- Bao nhiêu trạng thái khác nhau được tạo ra trong pha inital permuation ? Và nếu sử dụng một bộ đệm lưu trạng thái thì cần dung lượng là bao nhiêu bit?

Review

OTP: $E(k,m) = m \oplus k$, $D(k,c) = c \oplus k$

Making OTP practical using a PRG: $G: K \longrightarrow \{0,1\}^n$

Stream cipher: $E(k,m) = m \oplus G(k)$, $D(k,c) = c \oplus G(k)$

Security: PRG must be unpredictable (better def in two segments)

Attack 1: two time pad is insecure !!

Never use stream cipher key more than once !!

$$C_1 \leftarrow m_1 \oplus PRG(k)$$

$$C_2 \leftarrow m_2 \oplus PRG(k)$$

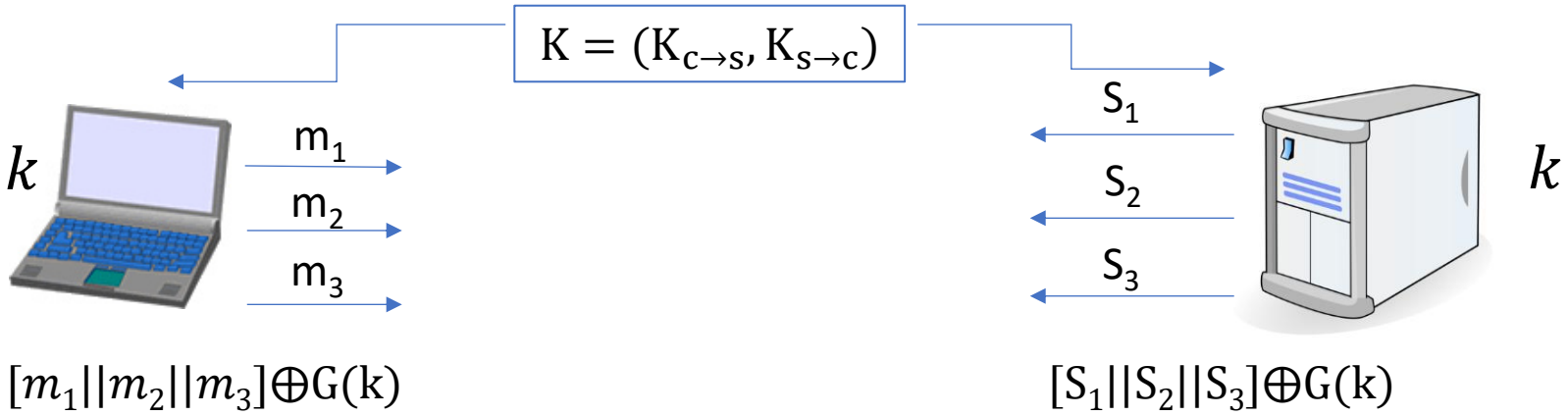
Eavesdropper does: $C_1 \oplus C_2 \rightarrow$ $m_1 \oplus m_2$

Enough redundancy in English and ASCII encoding that:

$m_1 \oplus m_2 \rightarrow$

Các ví dụ thực tế

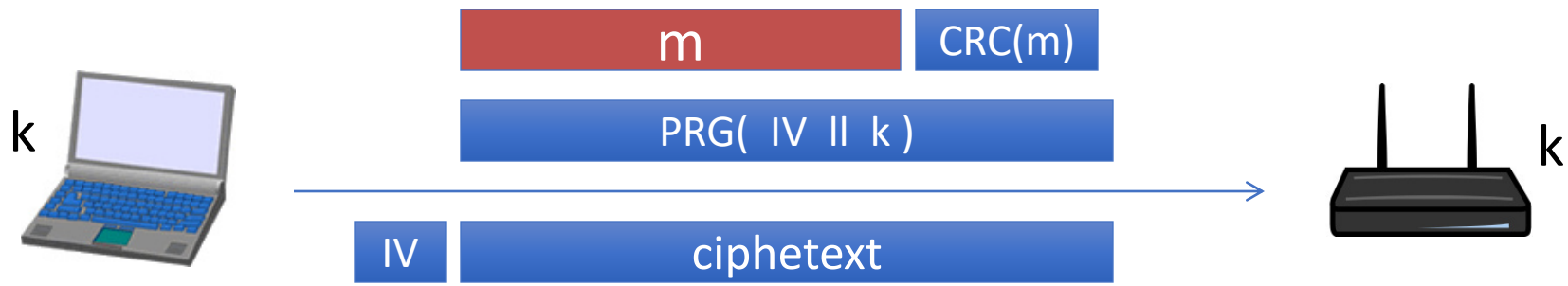
- Project Venona
- MS-PPTP (windows NT):



Need different keys for $C \longrightarrow S$ and $S \longrightarrow C$

Real world examples

802.11b WEP:

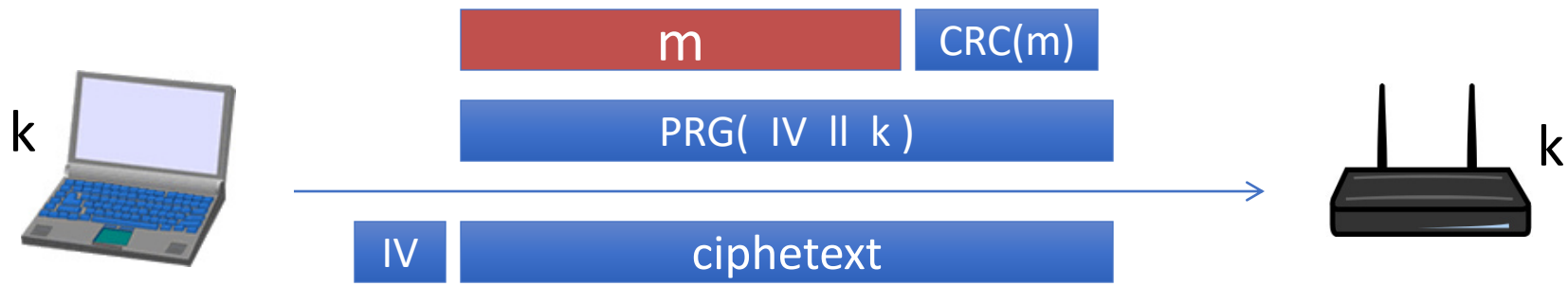


Length of IV: 24 bits

- Repeated IV after $2^{24} \approx 16M$ frames
- On some 802.11 cards: IV resets to 0 after power cycle

Avoid related keys

802.11b WEP:



key for frame #1: (1 ll k)

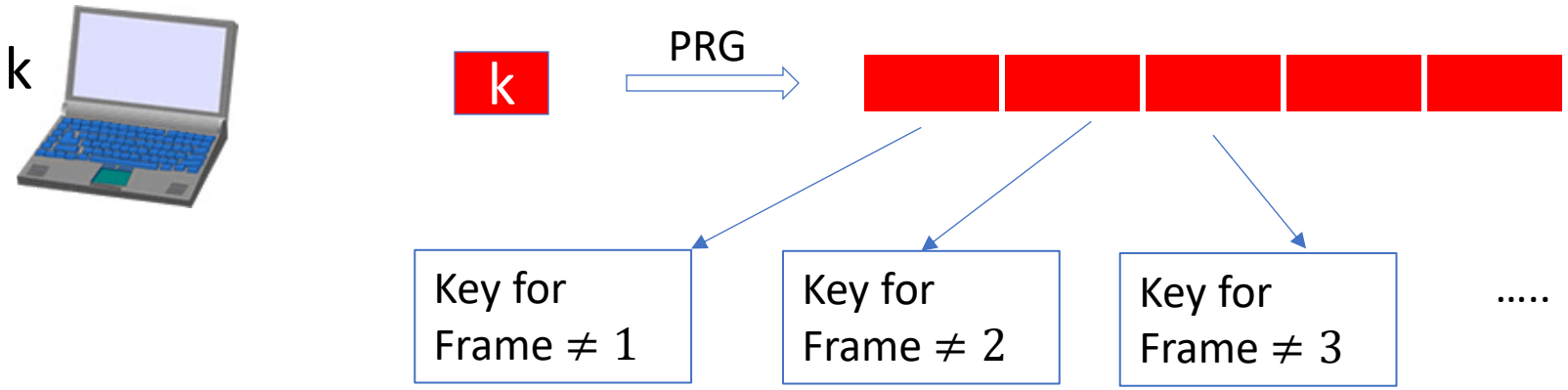
key for frame #2: (2 ll k)

24 bits 104 bits

⋮

For the RC4 PRG:
FMS 2001 -> can recover K after 10^6 frames Recent attacks ≈ 40000 frames

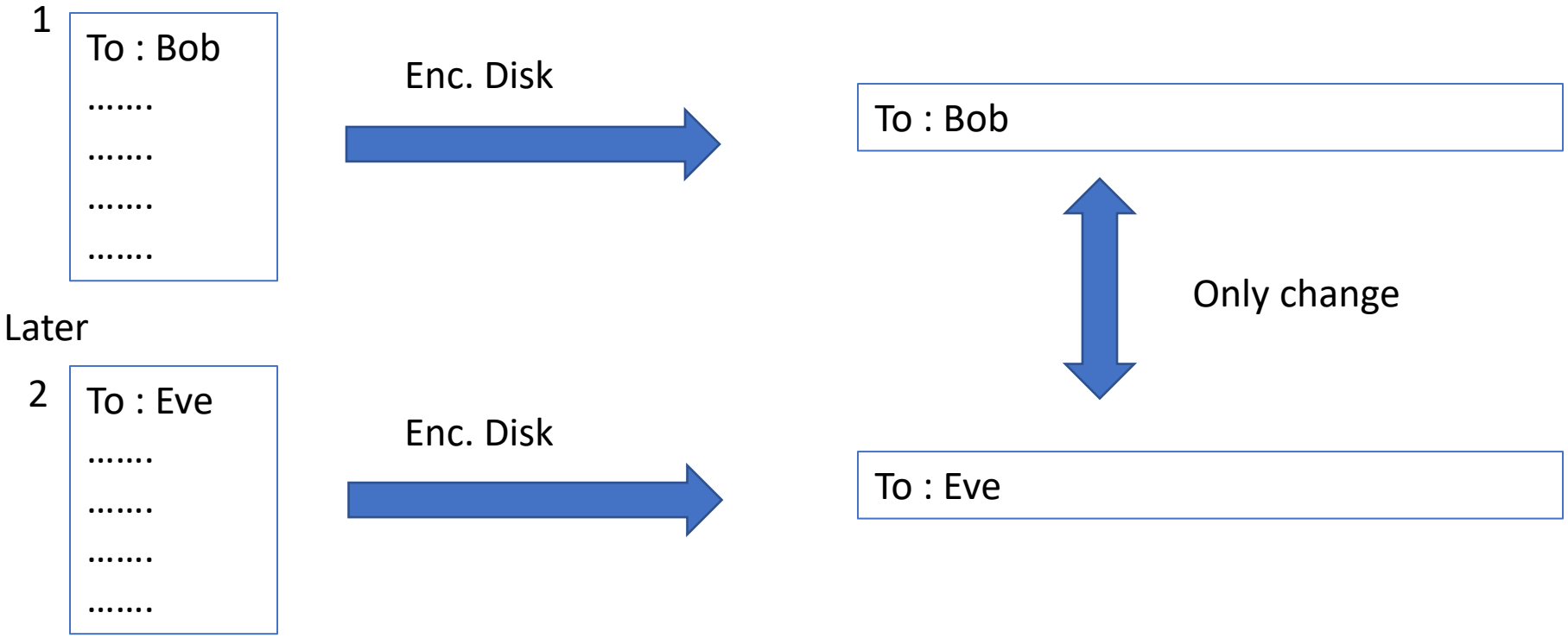
Cấu trúc tốt hơn



⇒ now each frame has a pseudorandom key

better solution: use stronger encryption method (as in WPA2)

Mã hoá ổ cứng (một ví dụ khác)



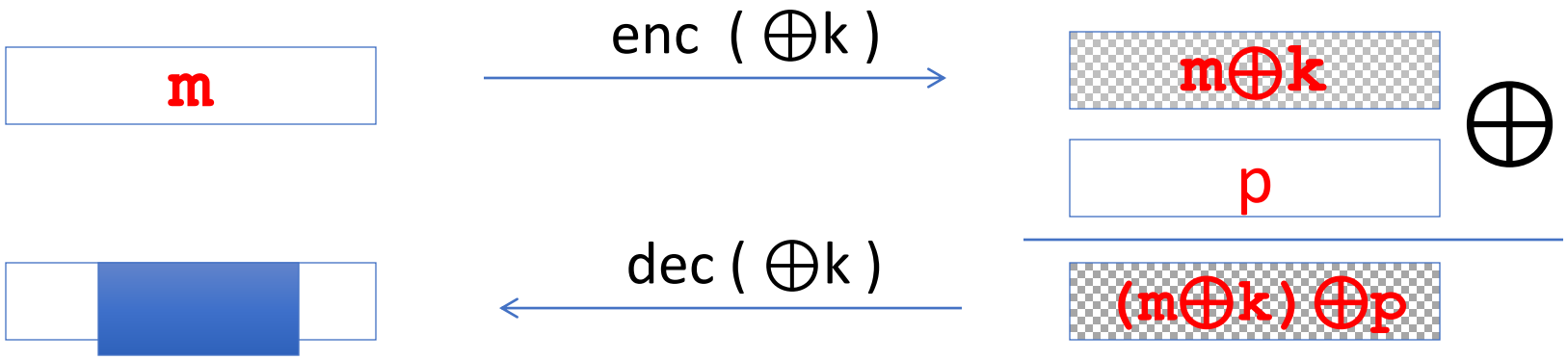
Two time pad: summary

Never use stream cipher key more than once !!

- Network traffic: negotiate new key for every session (e.g. TLS)
- Disk encryption: typically do not use a stream cipher

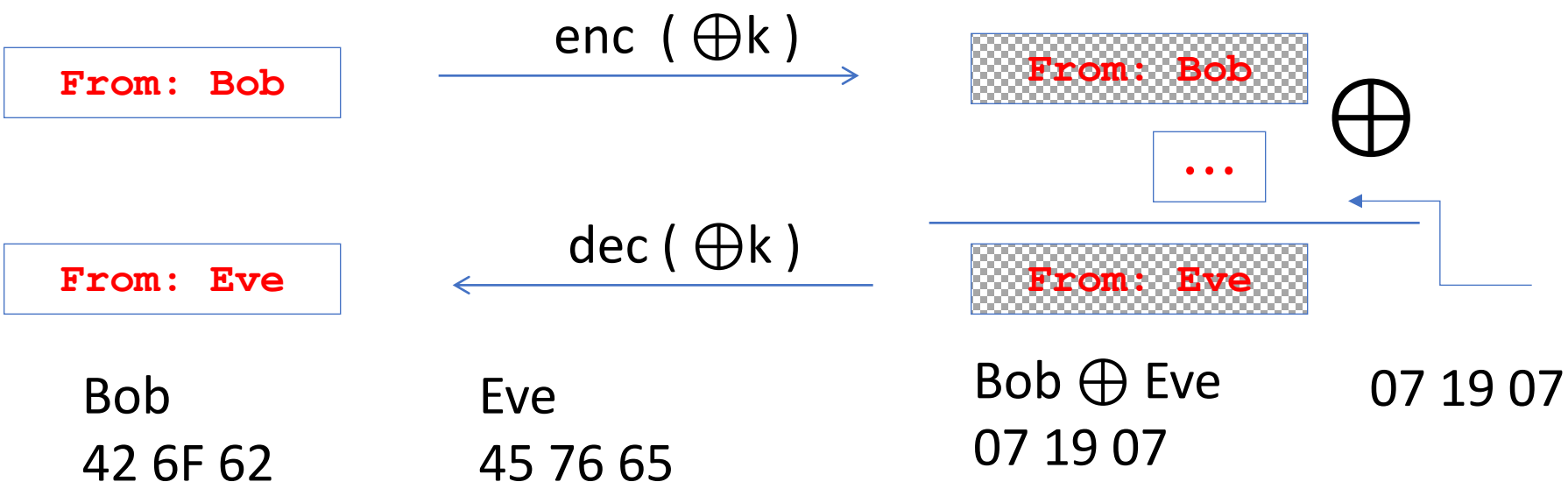
Attack 2: no integrity (OTP is malleable)

Modifications to ciphertext are undetected and have **predictable** impact on plaintext



Attack 2: no integrity (OTP is malleable)

Modifications to ciphertext are undetected and have predictable impact on plaintext



Câu hỏi về RC4

- Alice and Bob thoả thuận bí mật với nhau qua email sử dụng lược đồ mã hoá RC4, nhưng muốn tránh việc sử dung một khoá mật mới cho mỗi lần truyền. Alice và Bob thoả thuận bí mật với nhau một khoá k 128-bit. Để mã hoá một bản tin m bao gồm một chuỗi các bit, và được thực hiện như sau
- Chọn ngẫu nhiên một giá trị v 80-bit
- Generate the ciphertext $C = \mathrm{RC4(v||k)} \oplus \mathrm{m}$
- Gửi đi chuỗi bit $(v \,||\, C)$

Câu hỏi:

Giả sử Alice sử dụng cách này để gửi đi bản mã của m cho Bob, mô tả các Bob khôi phục lại m từ chuỗi $(v \,||\, C)$

Nếu một kẻ địch (adversary) quan sát được giá trị các cặp (v1 || C1), (v2 || C2), ... Truyền qua lại giữa Alice và Bob, bằng cách nào để kẻ địch nhận diện được khi nào khoá dòng được bị lặp lại cho việc mã hai bản tin

Câu hỏi về RC4

- Sấp xỉ khoảng bao nhiêu tin nhắn Alice có thể gửi đi được trước khi khoá dòng bị lặp lại? Số lượng bản tin có thể được mã hoá an toàn bởi k

Bài toán ngày sinh (The birthday paradox)

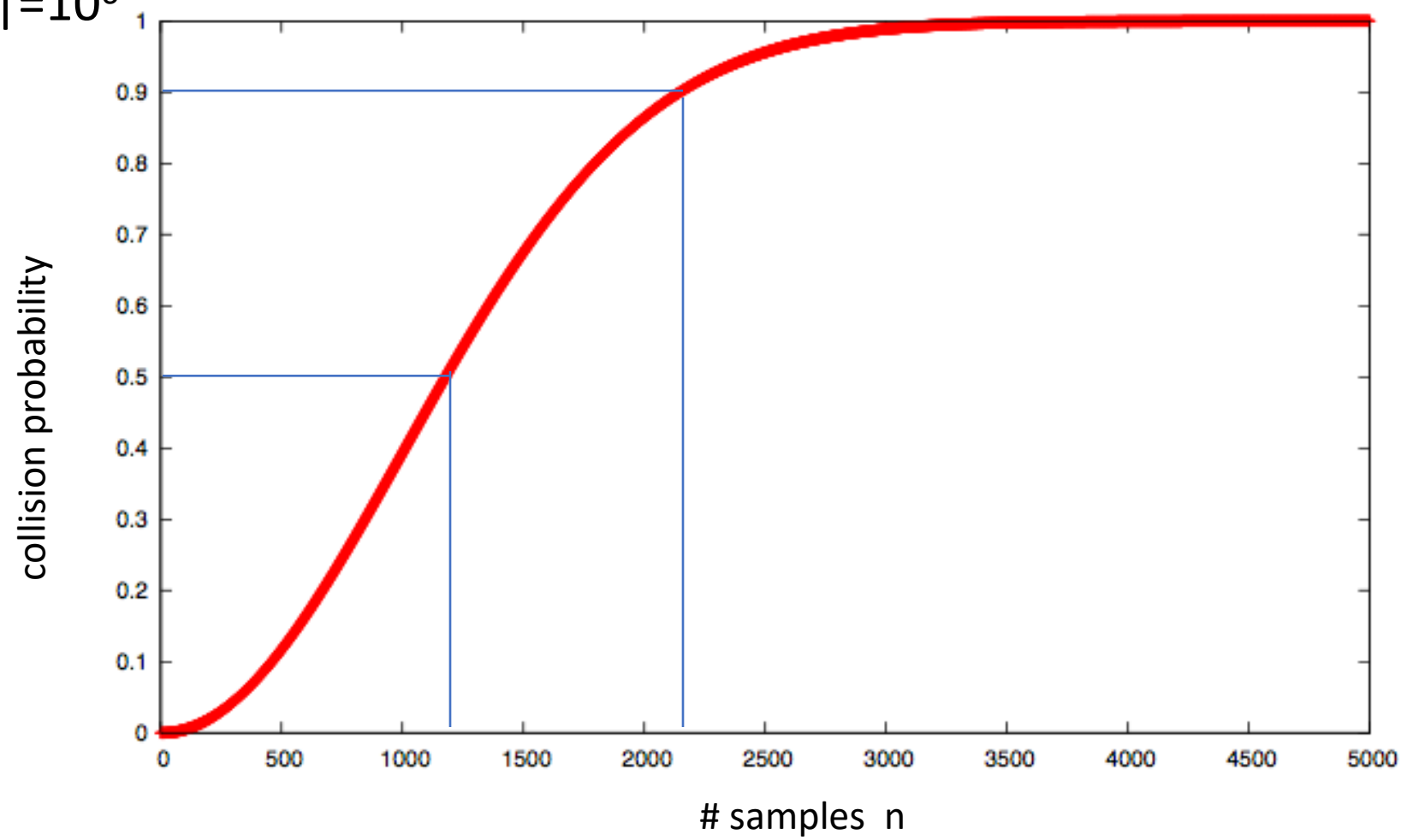
Let $r_1, \ldots, r_n \in U$ be indep. identically distributed random vars.

Thm: when $n = 1.2 \times |U|^{1/2}$ then $\Pr[\exists i \neq j: r_i = r_j] \geq ½$

Example: Let $U = \{0,1\}^{128}$ notation: $|U|$ is the size of U

Sau khi lấy mẫu khoảng 2^{64} bản tin ngẫu nhiên từ U, sẽ có một vài 2 cặp mẫu sẽ gần như tương tự nhau

$|U|=10^6$


collision probability
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
0
500
1000
1500
2000
2500
3000
3500
4000
4500
5000
samples n